Operating TriageNorth

Updated 22 October 2026. The native portal is http://127.0.0.1:38081. Use the existing portal login. Application/API identity stays intelligent-operations.

Daily operator screens

Start in **Overview** for quarantined events, retry/attention states, source receipts and canonical incident work. **Systems** puts the captured resource tree beside its selected resource and current-user pod diagnostics. **Sources** provides search, exact revision/mapping details and source-specific links into the pipeline; registration and test import are separate controls.

Data pipeline separates Flow, Events, Rules, Automation and Correlation. Events has source and validation filters, search, explicit selection and a readable decision trail. Retry/conflict receipts and the original payload remain inspectable. Pipeline tabs support arrow keys; the same panels stack at phone widths. Counts refer to the returned page and captured health retains its observation time.

In **Automation**, explicitly enable retained-evidence capture and choose source revisions before activating the existing personal schedule. A matching capture rule can then preserve already retained, resource-scoped context in an immutable packet. This does not collect new infrastructure data, start an AI investigation or execute remediation. The native scheduled acceptance used five synthetic occurrences: four matches became one admitted symptom, three suppressed duplicates and one automatically captured packet; the fifth event did not match. After acceptance, processing and capture are disabled, the temporary source scope is cleared and the schedule is inactive.

See [the daily operator guide](OPERATOR-WORKSPACE.md) for the screen-by-screen journey. Systems retains readable cluster names while filtering and uses compact desktop filters. For expiring Kubernetes connections, follow [the pod console renewal steps](POD-CONSOLE.md#expiring-connections); saved observations alone do not establish current access.

Adjust percolation queries

Open **Data pipeline → Rules**. Choose the named source revision, then edit its policy name, rules, enable/capture choices and fixed/sliding suppression window. Add AND conditions with the field/value controls; use each rule's stored-query JSON for nested bool, match, terms or explicit time ranges. Example: require `status=firing`, add `severity` is one of `["error","critical"]`, and request context for the matching rule. Unsupported query operators are refused by the real detection backend rather than silently treated as no match.

In **Events**, select 1–20 recorded alerts from that same source, then open **Rules → Test selected events**. The result shows which rules matched each actual delivery. Testing saves an immutable candidate; it changes neither the active source binding nor incident counts or capture reservations. **Publish and activate** is the separate live change. Earlier decisions remain visible with their original generation. Saved generations can be loaded as drafts for a reviewed rollback; stale source revisions are refused.

Understand the two filtering layers

Prometheus uses **PromQL** to turn numeric observations into source alerts. The current lab defaults scrape and evaluate every five seconds. `LabApplicationErrors` measures 5xx request rates above 0.1 requests/second for ten seconds; readiness and restart rules observe actual Kubernetes state. The named source's **OpenSearch percolation queries** then decide which admitted alerts proceed to reduction and evidence. These are separate queries with separate version/provenance records.

Alertmanager groups by alertname/environment, waits five seconds for a group, has a 15-second group interval and one-minute repeat interval, and sends resolved deliveries. Grouping does not replace source/type/native-resource symptom keys. The original lab receiver is `http://event-recorder:8080/webhook`; continuous native source admission is being connected separately. Do not interpret recorder-only deliveries as product tickets. Maintained baseline configuration is in `lab/lab.py`; the live Kubernetes ConfigMaps are the authoritative installed configuration. `lab/verify.py` exercises live HTTP failure → firing → webhook → recovery → clearing → distinct scaled pod UID.

Recorded datasets

The verified optional cache is `~/.cache/triagenorth/datasets`. `lab/datasets.json` pins commits, hashes, file sizes and upstream notices. Prepare/verify it with `python3 intelligent-operations/lab/prepare_datasets.py`. It contains 2,000 sampled raw lines each from BGL, OpenStack, Linux and HDFS, plus their selected parsing inputs and NAB EC2 latency/Auto Scaling CPU series and normal controls. Loghub research notices are retained and these inputs are separate from the redistributed application bundle.

`node intelligent-operations/lab/evaluate_recorded.cjs` runs the existing opt-in BGL severity baseline against explicitly disposable PostgreSQL/OpenSearch fixtures. Its measured receipt is `deploy/evidence/recorded-bgl.json`: 35.48% precision and 260 false positives, with labels excluded from query inputs. The native synthetic datacenter replay is a separate regression demonstration. Neither is a calibrated anomaly engine.

Live fault work

The owned cluster has one control plane and two workers. The 21 October supervised baseline run proves actual application errors, Prometheus firing, Alertmanager delivery, HTTP recovery, alert clearing and a new scaled pod UID. It restores two healthy API replicas and removes the load Jobs. Third-party software patch/configuration/crash scenarios and the connected automatic investigation/approved-recovery path remain active implementation work; this guide will record their actual commands and receipts as those paths pass, rather than marking them complete from the baseline alone.

3D graph presets

Open **Topology**. Load a graph at the desired ISO timestamp and optional collection, select a focus asset and choose a predefined query. **Dependencies of focus** follows outgoing dependency/routing edges; **Impact on dependents** follows the reverse direction. These paths show potential impact, not proven causation. **Placement of focus** follows recorded placement/ownership. **Observed degraded assets** excludes unknown health. **Observations older than 15 minutes** compares capture time with the selected snapshot time.

Drag to orbit, Shift-drag to pan, use wheel or buttons to zoom, or use arrow keys on the canvas. The selectable node list offers the same asset inspection with a keyboard. The Display selector retains the 2D view. Counts disclose 100-node/500-edge display limits and a 500-identity traversal limit. Depth is a layout by source scope, not measured network distance. Refresh or access withdrawal clears the private view.

The rule workbench at source7ed15232 was installed through native trust/preview/consent and passed actual HTML sign-in, candidate1/3 matching, unchanged active binding/decisions/reservations during trial and explicit binding revision2 activation. See policy-workbench.json. This does not certify later workflow or external provider delivery.

Investigation handoff and funnel

In **Data pipeline → Flow**, the existing cohort funnel shows unique evaluated events, rule matches, deduplication survivors and recorded correlation groups. Below it, **From evidence to an accepted report** links those same delivery IDs to pinned packets and canonical workflows. Expand **Evidence work** and click **Start AI investigation** on a capture reservation. The application pins a bounded preprocessed packet; the native workflow receives only that fixed file and its digest. It cannot inspect raw telemetry or the cluster. Refresh workflow progress to read actual completed native outputs and inference receipts. Open the canonical ticket for its workflow, shared workspace and report approval.

Retrying an already linked reservation resumes its existing ticket. An uncertain creation remains held and says to reconcile the canonical board; it never silently creates a second ticket. Done means report accepted. Verified infrastructure recovery stays **Not connected** until a fresh check is implemented. The workflow currently produces its accepted answer.md report; this handoff does not certify generated executable remediation bundles.

SMS service support is now committed in the separate kernel at92da94c, with actual loopback-TLS, native permission and installer checks. Full kernel gate and installation are still pending. Environment key presence was inspected without printing values; no live SMS/call/Telegram transmission was performed. Calling Assistant already has a separate guarded carrier path. Jira/GitLab status-transition catalog gaps and durable synchronization remain under active build.

Native acceptance now confirms the3D presets and real workflow handoff at24d62980. The accepted first-phase report is available in ticket ca7fc196-8a0b-4055-809d-7b957cde68f9 and its shared workspace. deploy/evidence/operating-experience.json records the exact source, artifact, input/report hashes and inference receipt. The underlying evidence is explicitly synthetic; model execution and shared-workspace output are genuine.

The fitted perspective view at source a8697163 is installed and rechecked with `node intelligent-operations/deploy/operating-acceptance.cjs --graph-only`. This uses the original graph's exact observation timestamp, avoiding a false stale result as wall time advances, and dispatches no new inference.

`--workflow-only` reads the existing acceptance ticket and downloads its actual shared-PVC outputs. All three investigate/review/revise phases now have completed answers and inference receipts; all three downloaded artifact sizes and SHA-256 hashes match workspace metadata. The ticket remains awaiting human report approval. See deploy/evidence/workflow-artifacts.json and graph-experience.json. This verification does not generate another ticket or approve the report.

Check connections and messaging standing

Open **Connections & tools → Refresh my connections**. Personal account liveness, selected HTTP services and your invocation scope appear separately. An older kernel without service metadata explicitly requests the connector rollout. Registered SMS, Telegram and email transports are compared with your enabled topic routes without showing destinations. A registered transport is not a successful delivery test. This screen makes no sends or provider actions and clears private content after refusal. Brief phone alerts require the explicitly enabled shared voice transport; interactive Calling Assistant retains its separate account/approval path.

For a specifically authorized local acceptance test, the operator-side `lab/communications.py` uses the installed portal's standard notification API. Select the existing literal destination file and owned workflow ticket, then pass `--authorized-once` only for the approved one-SMS, one-brief-call, one-Telegram test. It preflights the selected transports before sending, saves and fsyncs a private reservation before each possible send, and mutes its own test topics afterward. Reopening the runner never resends an attempted channel, including a lost response. Never delete or reset its private ledger to force a retry. Inspect an uncertain outcome before requesting any additional test. Carrier acceptance does not prove handset delivery, answered speech or human receipt. The fixture tests send nothing. An explicit `--channel telegram` can finish that approved channel while another transport rolls out; the same ledger prevents a later all-channel run repeating it.

The local development installation now registers SMS, short phone alerts and Telegram. The authorized tests each have exactly one attempt and all test topics are muted. SMS and phone returned explicit carrier acceptance; Telegram returned success on the older portal contract and its initial response-version mismatch was reconciled without a resend. The operator confirmed the phone call. A read-only provider lookup confirms the SMS was undelivered with Twilio error30034 (unregistered US A2P sender); Telegram human receipt remains unverified. See `deploy/evidence/communications-acceptance.json`. General automatic incident notifications are not enabled by this one-time test approval.

Link a workflow step to an external ticket

In **Connections & tools → External ticket handoff**, refresh workflow links. Configure an immutable link to your existing Jira issue, GitLab issue or ServiceNow incident. Choose the canonical status and phase, then explicitly map the remote transition/state. **Load available Jira transitions** reads the issue's actual transition IDs and destination states. No default transition numbers are supplied. GitLab uses numeric project ID/issue IID and close/reopen; ServiceNow needs the instance state value and expected displayed value. **Save link only** sends nothing.

Review mapped update shows the exact proposed operation and workflow revision. **Confirm and push status** checks the service realm/account, reserves the snapshot, checks the canonical revision and uses the shared confirmed connector action. It reads remote state first; an already matching state needs no update. After a write, a new read checks the mapped state. The durable entry is your operator report, not a server-attested delivery receipt. It never changes canonical ticket status or proves infrastructure recovery. A lost response holds the attempt. Use **Read remote state** to reconcile; it sends no transition and never blindly repeats a write.

Requires the selected HTTP services in core ADR187. This manual workbench does not activate unattended two-way synchronization, webhook handling or external ticket creation. Splunk On-Call and GitHub handoff remain separate provider work.

The selected Jira site and original-person primary account are configured in the development installation. Its read-only profile probe returns HTTP 404, so account access/site validation remains open. No external issue has been changed. The native UI/schema and isolated confirmed-push tests do not prove live Jira synchronization. The development kernel is source `ac5f00e`; full clean-commit certification remains open, including the large artifact stream expiry found in the wider gate.

Inspect the evidence behind a bot decision

After the ADR190 kernel update, open the canonical ticket's Process or Workflow tab. The graph marks the current phase and approval gate. Each phase names its bot and shows its admitted input digest, task text and individual file hashes/content. The file lineage counts added or changed inputs, including forwarded phase reports. Open **Data presented to this bot** to inspect the actual submitted packet. Text omitted by the bounded projection is explicitly labelled; no raw cluster authority is exposed. **Open this ticket in Workflow Studio** opens the same runtime graph read-only and creates no draft. Use normal Studio authoring to edit a workflow.

Cost shows recorded usage and separately labelled reference estimates for calls recorded at zero. The Kubernetes budget-reference assumption is$1 input/$3 output per million tokens, configurable through the two cost-estimate ConfigMap settings. These assumptions are not billed provider prices. Missing rates show Unpriced; sub-cent estimates remain visible. Current-day filtering does not create tickets: only admitted evidence reservations sent to the queue appear as investigations.

Create is the unified studio home; AI Office is its document/deck component. For mailbox setup use `/utilities?category=email`. Portal email alerts separately need an administrator SMTP relay or webhook transport. Connecting Outlook/Gmail alone does not register that transport or send a test message.

Approve and run a generated response, then verify recovery — 22 October

Open Incident work → Approved response and verified recovery. Refresh response plans, then expand "Prepare a response plan from accepted work": choose a ticket at its report gate, the completed phase with the accepted answer (the page previews which generated files it carries: `remediate.sh`, `verify.sh`, `rollback.md`, `verify.md`), the observed target pod from your inventory, its container and your Kubernetes account, and prepare the plan. Download and read the files; their digests are shown. Approve this exact action (thirty-minute expiry, bound to the digests, the target and the canonical revision). Execute approved action runs the script once through your own Kubernetes connection with a fresh confirmation. Verify recovery runs `verify.sh` and a pod readback; recovery is healthy only with a passing check, the same pod identity and a resolved alert admitted after the action. Only then does Accept report and update ticket approve the canonical gate. The execution receipt is downloadable. On the lab this closed ticket 46b8df31 at revision 5 (`deploy/evidence/response.json`).

Promote a response through Git, or reconcile a break-glass change

On a response plan, expand Promote through Git. Choose GitHub or GitLab, your own account, the repository (owner and name, or the numeric GitLab project), the base branch, a folder in the repository and, for GitHub, an optional workflow file; tick Start the pipeline to run it on the new branch. Commit, open request commits the plan's exact files and a manifest to `triagenorth-<plan>` and opens a pull or merge request; every provider write asks for a fresh confirmation. Use this when the pipeline should apply the change. If you executed the plan as an emergency change, Reconcile break-glass change in Git appears: it commits the same files with the execution receipt to `triagenorth-reconcile-<plan>`, and the request must be merged or reverted in Git within 24 hours; the panel marks it overdue after that. Refresh status reads merge and pipeline state. The GitHub or GitLab service must be selected by an administrator and your account connected first.

Detect anomalies on your own telemetry

Open Live context → Anomaly detection. Choose the telemetry stream sources to score, the alert source that should receive candidates (publish a percolation policy for it under Data pipeline → Rules so candidates become decisions), the detector (rolling robust z-score, or the seasonal time-of-day profile for daily patterns) and its parameters, then enable scoring. Each scheduled tick scores new metric samples before learning from them; the panel shows per-resource readiness (cold, ready, stale, late) with the last score and baseline, and Run detection now scores pending samples immediately. Candidates appear in Events as `anomaly:<detector>:<metric>` and follow the same filtering and noise reduction as source alerts. The NAB pilot receipt (`deploy/evidence/anomaly-nab.json`) records measured detection, lead time and false-alert rates for both detectors; it is not an official NAB score.

Read what is in each cluster node

Open Systems and expand a Kubernetes system. Nodes are listed with the namespaces and workloads; selecting a node shows Node facts: capacity against allocatable (CPU, memory, pods), pressure conditions and cordon state, roles, taints, internal IP and hostname, kubelet, OS, container runtime, architecture and kernel, and the pods placed on the node. Each pod is a link that opens that pod in the tree, where the pod console offers current status, logs and reviewed commands through your own connection. Facts come from the imported scan (`lab/topology.py --output` then Topology → Import a source-owned scan); absent facts are omitted rather than fabricated.

Get a command coached and run it under your own authorization

Open Command helper. Pick the tool (kubectl, AWS CLI, Google Cloud CLI, Azure CLI, or inside a pod container), say where you will run it, describe the goal and any context, and ask the operations analyst. It proposes one command as explicit arguments with its reason, whether it is read-only, its risk and how to verify the result. For an in-pod command, choose the observed pod, container and your Kubernetes account and run it through your own connection with a fresh confirmation; the result is recorded. For kubectl and cloud CLIs, copy the command, run it in your own terminal or OSHAL Node under your signed-in profile, paste the output and record it; the record says it is operator-provided. The application never holds your cloud credentials and never runs a cloud command itself; in-application cloud execution needs the kernel rails listed in KERNEL-RAILS-NEEDED.md.

Installed decision and mailbox checks — 21 October

The installed 6f1c38f development kernel passed actual HTML password sign-in, original-owned input/receipt visibility, named bot attribution, separately labelled reference cost and desktop/phone Studio runtime rendering. The baseline used an existing real three-phase ticket; earlier reports are never presented as satisfying new instructions retroactively. Create is installed from public application bceffaf0 and opens at `/cockpit/?app=create`, preserving the existing saved Office work.

Use `/utilities?category=email`, then **Microsoft 365 (Outlook)** or **Gmail**. The Microsoft client has been prepared through the reviewed private installer; authorization URL generation works. The signed-in person must still complete provider consent, and the provider must allow the installation's callback. Portal notification delivery needs an administrator SMTP/webhook transport separately. Neither mailbox consent nor an actual email delivery has been certified.

The bounded local reproduction can be run with `python3 lab/verify.py`, followed by `python3 lab/decision_incident.py` from the application folder. The second command admits one recent, hash-checked monitoring capture and reserves before mutations. It refuses a second episode while its durable receipt exists. It does not give the agent access to Kubernetes or the source stream.

The fresh ticket 2a60f877 failed at the account receiver's 45 second deadline. Its failed state is retained. The central782ae3d deadline correction uses90 seconds for account inference,105 seconds for the kernel request and the unchanged120 second phase limit. `python3 lab/decision_retry.py` performs one explicitly requested linked child attempt against the same sealed packet; it also reserves before creation and refuses duplication. Its receipt labels prior provider consumption unknown when no successful usage receipt exists. Do not delete either receipt to force retries after uncertain outcomes.

Run `node scripts/operations-decision-acceptance.cjs TICKET_UUID` from the repository root only against a completed fresh three-phase investigation. It signs into the actual HTML portal and verifies the evidence/decision tables, retained earlier reports, reference cost, runtime graph, Create and mail category without approving a report or sending a notification. Use a bounded CPU/memory scope and keep GPU acceleration disabled as in the helper.

The installed linked retry 537d9d04 completed all 3 analysis phases and is waiting for human report approval. Its 3 workspace reports pass authenticated download/hash checks. Cost shows 39,166 tokens and a separately labelled $0.050048 reference estimate, with $0 recorded spend. This is a budget assumption, not billed cost. The sealed packet, original report and reviewer response all reach the revise phase. These are reviewable diagnostic/verification proposals; no infrastructure action ran and standalone

executable script files have not been materialized.